

Amazon ML Challenge 2026 — Top-3 Preparation Playbook

A practical pre-hackathon system for scenario prediction, reusable ML pipelines, team execution, validation, experimentation, and learning from previous Amazon ML Challenge solutions.

Objective: arrive on Day 1 with problem-agnostic engineering already built, a decision tree for unknown problem statements, and a team workflow that minimizes discussion time and maximizes experiments.

1. What previous challenges tell us

Amazon has repeatedly used real e-commerce/product problems, but the task type has changed. In 2024, the challenge focused on extracting product entity values such as weight, volume and dimensions from product images. A published Rank-6 solution used an ensemble of MiniCPM-2.6 and Qwen2-VL-7B with zero-shot and few-shot prompting; another published winning-code repository describes Qwen2-VL fine-tuning with QLoRA and data curation.

■cite■turn0search8■turn0search9■

In 2025, the task was Smart Product Pricing: predict product price from catalog text and product images, with 75,000 training and 75,000 test products and SMAPE as the metric. External price lookup was prohibited. ■cite■turn0search0■

The 2025 results show why experimentation matters. The Top-3 team 00_Team_Rocket reported that it removed image embeddings after finding them noisy and used a two-stage DeBERTa + cross-attention system with handcrafted numeric/binary features. ■cite■turn1search0■

A published 3rd-public/5th-private solution instead used pretrained multimodal embeddings, modality-specific networks and a final regressor, showing that multimodal representations can work without fine-tuning the large encoders.

■cite■turn0search1■

Another published solution ended with a lightweight LightGBM pipeline using TF-IDF, quantity features, log-transformed targets and K-fold validation after testing more complex approaches. ■cite■turn0search2■

A reproducible Top-7% pipeline combined SigLIP2 image/text embeddings, MiniLM, engineered catalog features, LightGBM, XGBoost, CatBoost and an MLP with OOF blending. ■cite■turn0search3■

2. Core strategy

Do not prepare one giant model for a guessed problem. Prepare reusable pipelines and a decision tree. The historical pattern is not 'Amazon always asks price prediction'; it is 'Amazon asks a concrete product/e-commerce ML problem where extracting signal from messy product data matters.'

Prepare five families: tabular regression/classification, text + tabular, image + text, time-series/demand, and ranking/recommendation. These are preparation hypotheses, not predictions of the 2026 problem.

3. 15 plausible scenarios and first-choice models

Scenario	Task	Primary model family	Backup / ensemble
1. Product price	Regression	CatBoost / LightGBM	XGBoost + text model
2. Product sales	Regression	LightGBM	CatBoost + XGBoost
3. Demand forecast	Time-series regression	LightGBM + lag features	XGBoost / specialized TS
4. Bestseller prediction	Classification	CatBoost	LightGBM / XGBoost
5. Product rating	Regression	CatBoost	LightGBM + text encoder
6. Review-count prediction	Count/regression	LightGBM on log target	CatBoost / XGBoost
7. Delivery-time prediction	Regression	CatBoost	LightGBM
8. Order cancellation	Classification	CatBoost	LightGBM + calibrated probabilities
9. Return probability	Classification	LightGBM / CatBoost	XGBoost
10. Purchase/conversion probability	Classification	LightGBM	CatBoost
11. Product recommendation	Ranking	LightGBM LambdaMART	XGBoost Ranker

Scenario	Task	Primary model family	Backup / ensemble
12. Search relevance	Ranking	LambdaMART + embeddings	cross-encoder / reranker
13. Inventory requirement	Forecasting/regression	LightGBM + lag/rolling	XGBoost
14. Product category image+text	Multimodal classification	Embeddings + LightGBM/CatBoost	VLM / MLP fusion
15. Quality/return-risk image+text	Multimodal classification	Text+image embeddings + CatBoost	MLP fusion / boosting

These are starting points. The actual metric, data distribution, compute budget and validation design must determine the final model.

4. Pre-hackathon advantage: build the machinery, not the answer

Your idea is strong: pre-build everything that is problem-agnostic. After the problem statement drops, the team should activate modules rather than spend hours writing boilerplate.

- **Data loader:** CSV/Parquet ingestion, schema inspection, dtype inference, duplicate detection, missingness report.
- **EDA generator:** target distribution, numerical summary, categorical cardinality, missingness, correlations, train/test drift.
- **Cleaning:** missing-value policies, duplicate handling, type coercion, whitespace normalization, outlier diagnostics.
- **Text:** cleaning, token/character statistics, TF-IDF, SVD, sentence-transformer embedding interface.
- **Image:** URL downloader with retry/cache, corrupted-image detector, resize/normalization, batch embedding interface.
- **Features:** ratios, counts, lengths, frequency encoding, numeric extraction from text, unit parsing, date parts, group statistics.
- **Validation:** K-fold, stratified K-fold, group K-fold, time split, OOF prediction storage, metric registry.
- **Models:** CatBoost, LightGBM, XGBoost, linear baseline, MLP, ranking model.
- **Ensemble:** blend OOF predictions, optimize non-negative weights, compare calibration.
- **Experiment logger:** config, seed, features, fold split, CV score, runtime, GPU memory, prediction artifact.
- **Submission checker:** row count, ID alignment, duplicates, NaN/Inf, target range, required names, ordering.
- **Reproducibility:** requirements/environment file, fixed seeds, config YAML/JSON, one-command training/inference.

Boundary: do not precompute challenge-specific features from unknown future data or external sources. Prepare code and interfaces; activate them only when the released data/rules permit them.

5. Recommended repository structure

```
amazon-ml-2026/  
■■■ configs/ # baseline, text, image, ensemble  
■■■ src/  
■ ■■■ data/ # loaders + schema checks  
■ ■■■ eda/ # automated reports  
■ ■■■ cleaning/ # generic cleaning  
■ ■■■ features/ # tabular/text/image  
■ ■■■ validation/ # CV + OOF  
■ ■■■ models/ # LGBM/CatBoost/XGB/neural  
■ ■■■ ensemble/ # blending  
■ ■■■ submission/ # final validators  
■■■ notebooks/ # EDA/baseline/experiments  
■■■ artifacts/ # cached features + OOF  
■■■ submissions/  
■■■ run_pipeline.py
```

6. First 2–3 hours after the problem drops

- **0–15 min:** read the statement twice; extract target, allowed inputs, forbidden data, metric, submission schema, compute limits and deadline.
- **15–45 min:** automated EDA on train/test; identify target type, missingness, cardinality, duplicates, distribution, train/test drift and leakage risks.
- **45–75 min:** simplest valid baseline; save CV score and first submission.
- **75–120 min:** strongest cheap feature block: text TF-IDF/numeric extraction, tabular engineering, or pretrained image embeddings.
- **2–3 hr:** run 2–4 orthogonal models; compare CV, runtime and error patterns; choose the next experiments from evidence.

7. 72-hour execution plan

Time	Goal	Output
0–3 h	Understand + baseline	Validated baseline + first submission
3–8 h	Feature engineering	Strong feature blocks
8–16 h	Model sweep	2–4 competitive families
16–24 h	Error analysis	Failure categories + targeted features
24–36 h	Advanced models	Transformer/VLM/MLP only if justified
36–48 h	Ensemble	OOF predictions + blend candidates
48–60 h	Generalization	CV/test-gap + leakage + robustness checks
60–68 h	Final optimization	Stable final pipeline
68–72 h	Submission	Final file + report/code package

8. Team operating system

Role	Responsibility
Lead/integrator	Locks metric/validation, prioritizes experiments, owns final pipeline.
Data/features	EDA, cleaning, text/tabular/image features, drift/leakage.
Model engineer	Model families, hyperparameters, GPU jobs, error analysis.
Ensemble/infra	OOF, blending, submission checks, runtime, report/code packaging.

Communication rule: 10-minute start-of-day sync, then asynchronous updates in one experiment table. Every experiment records hypothesis → change → CV → leaderboard → decision.

9. Lessons from high-ranking teams

- **Validate before complexity:** the 2025 Top-3 team dropped image embeddings when experiments suggested they added noise. ■cite■turn1search0■
- **Simple can win:** the published FLANKERS solution finished with TF-IDF + handcrafted features + LightGBM after more complex trials. ■cite■turn0search2■

- **Ensemble complementary errors:** the Top-7% published pipeline blended LightGBM, XGBoost, CatBoost and MLP using OOF predictions. ■cite■turn0search3■
- **Multimodal needs architecture:** the 2025 3rd-public/5th-private team used modality-specific networks and a final regressor rather than simply concatenating everything. ■cite■turn0search1■
- **VLM/OCR matters when the task is extraction:** 2024 Rank-6 used MiniCPM-2.6 + Qwen2-VL-7B with zero/few-shot prompting; another winning-code repository used QLoRA fine-tuning and post-processing. ■cite■turn0search8■turn0search9■
- **Pivot quickly:** one 2025 Top-22 account reported moving from LightGBM/XGBoost to RoBERTa and then DeBERTa-V3-large, while dropping a multimodal setup that did not help. ■cite■turn1search1■

10. Model-selection matrix

Data	Start	Then test	Ensemble
Mostly tabular	CatBoost + LightGBM	XGBoost / ExtraTrees	Boosting blend
Text + tabular	TF-IDF + LightGBM	MiniLM/DeBERTa	Sparse + dense
Image + tabular	Embedding + CatBoost	CLIP/SigLIP + MLP	Blend modalities
Text + image	Separate embeddings	VLM / cross-attention	OOF weighted blend
Time series	Lag/rolling + LGBM	XGBoost / TS model	Horizon blend
Ranking	LambdaMART	XGBoost Ranker	Neural reranker
OCR/entity extraction	OCR + rules	VLM/VQA	Voting + post-process

11. Top-3 optimization priorities

- Speed: first valid submission early.
- Reliable validation: prevent leaderboard overfitting.
- Orthogonal experiments: every run tests a meaningful hypothesis.
- Compute efficiency: expensive training only after evidence of signal.
- Complementary models: optimize blends on OOF predictions, not leaderboard guesses.
- Robustness: ID alignment, missing/corrupt data, drift and reproducibility.
- Presentation readiness: maintain an architecture diagram and experiment history.

12. Pre-hackathon build checklist

- Repository + environment lockfile
- Automated train/test schema report
- Missing-value + duplicate report
- TF-IDF + SVD module
- Sentence-transformer embedding module
- Image downloader/cache + embedding module
- Numeric extraction from text
- CatBoost/LightGBM/XGBoost wrappers
- K-fold / stratified / group / time split wrappers
- OOF prediction saver
- Blend-weight optimizer
- Submission validator
- Experiment logger
- One-command baseline runner
- Synthetic end-to-end pipeline test

13. Exact decision protocol when the PS drops

1. Identify target + metric + constraints.
2. Classify task: regression / classification / ranking / extraction / forecasting.
3. Identify modalities: tabular / text / image / time.
4. Pick the matching baseline.
5. Establish CV before optimization.
6. Submit baseline early.
7. Add one modality/feature block at a time.
8. Keep only changes that improve reliable CV and/or generalize to leaderboard.
9. Store OOF predictions for every serious model.
10. Blend only complementary models.
11. Freeze final pipeline early enough for full-system testing.
12. Run final integrity check.

14. Bottom line

The competitive edge should not be 'we know the 2026 problem in advance.' It should be 'whatever Amazon gives us, we can turn the raw dataset into a validated baseline in under an hour and run disciplined experiments immediately.'

Historical evidence supports preparing for product/e-commerce ML, but the task has changed substantially across years: 2024 centered on image entity extraction while 2025 centered on multimodal price prediction. A modular toolkit + decision tree is therefore safer than a hard-coded price pipeline. ■cite■turn0search6■turn0search8■turn0search0■

Sources / further reading

Amazon ML Challenge 2025 — Top-ranking solution
<https://github.com/NeelDevenShah/Amazon-ML-Challenge-2025>

2025 — Team SPAM_LLMs (3rd public / 5th private)

<https://github.com/RudrakshSJoshi/amlc-multimodal-mlp>

2025 — Team FLANKERS solution

<https://github.com/vavaibhav31/Amazon-ML-challenge-2025>

2025 — reproducible multimodal Top-7% solution

<https://github.com/Spkap/amazon-ml-challenge-2025-Multimodal-Product-Price-Prediction>

2024 — Team DBkaScam (Rank 6)

<https://github.com/arnav10goel/Amazon-ML-Challenge-24>

2024 — winning-code / NeuralNinjas repository

<https://github.com/Stefan-431/Amazon-ML-Challenge>

2024 — Top-50 solution

<https://github.com/hwaseem04/Amazon-ML-Challenge-2024>